51. Agents and Tools: Function-Calling Loops

51. Agents and Tools: Function-Calling Loops

Phase: Phase 7 — AI engineering in Go
Estimated time: 2 days
Slug: `51-agents-tools`

Learning objectives

- Implement tool schemas
- Run a bounded agent loop
- Sandbox dangerous tools
- Enforce max steps and budgets

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Agents loop: model proposes a tool call → your code runs it → result returns to the model. Go's type safety helps keep tools honest.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Tools are typed functions with JSON schemas.
- Loop with a max step count to prevent runaway costs.
- Validate tool args before execution.
- Sandbox dangerous tools (shell, network).
- Log each step for auditability.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-agents && cd /tmp/gw-agents
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```go
import "fmt"

type Call struct {
    Tool string
    Arg  string
}

func RunTool(c Call) string {
    if c.Tool == "echo" {
        return c.Arg
    }
    return "unknown"
}

func main() { fmt.Println(RunTool(Call{Tool: "echo", Arg: "hi"})) }
```

Run it:

```
go run .
```

Expected output:

```
hi
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Dispatch a tool by name.
2. Stop after N steps even if model continues.
3. Reject unknown tools with an error string.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Agent loop

```
for step := 0; step < maxSteps; step++ {
  decision := llm.Next(history)
  if decision.Type == "final" { return decision.Answer }
  result := tools.Run(decision.Tool, decision.Args)
  history = append(history, result)
}
```

Safety

- Allowlist tools
- Validate args against JSON schema
- Timeouts per tool

- Human approval for side-effecting ops
- Hard caps on steps/tokens/cost

Go's type system and explicit errors help keep tool runners boring and safe.

Failure modes to watch while coding

- Unbounded loops
- Executing shell from model output unchecked
- Trusting model JSON without validation

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- LangChain agents ≈ same loop.
- Dynamic getattr tool calling is riskier — Go maps of funcs are clearer.
- Same need for hard timeouts.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Treat tool results as untrusted input when feeding back to the model.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Agents and Tools: Function-Calling Loops" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Unbounded loops
- Executing shell from model output unchecked

- Trusting model JSON without validation

Exercises

Work in `exercises/51-agents-tools/`. Read that folder's `README.md` before you code.

What you will build

Implement the agent loop/tool dispatch stubs.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/51-agents-tools/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Why maxSteps?
2. What to validate before tools.Run?
3. Allowlist purpose?

Further reading

- https://go.dev/blog/error-handling-and-go